

My Course Title

AUTHOR NAME

TABLE OF CONTENT

1. Аргументы командной строки.....	3
2. Циклы.....	3
а. Цикл for.....	3
b. Цикл while.....	3
3. Способы объявления.....	4
4. Поиск повторяющихся строк.....	5
5. Выборка URL.....	7
6. Параллельная выборка URL.....	8
7. Веб-сервер.....	9

My Course Title

1 – Аргументы командной строки

Первый элемент `os.Args`, `os.Args[0]`, представляет собой имя самой команды; остальные элементы представляют собой аргументы, которые были переданы программе, когда началось ее выполнение

```
package main

import (
    "fmt"
    "os"
)

func main() {
    var s, sep string
    sep = " "
    for i := 1; i < len(os.Args); i++ {
        s += sep + os.Args[i]
    }
    fmt.Println(s)
}
```

Но можно и так:

```
package main

import (
    "fmt"
    "os"
)

func main() {
    s, sep := "", " "
    for _, arg := range os.Args[1:] {
        s += sep + arg
    }
    fmt.Println(s)
}
```

2 – Циклы

a – Цикл for

```
for инициализация; условие; последствие {
    // тело цикла
}
```

b – Цикл while

```
// Традиционный цикл while
for условие {
    // тело цикла
}
```

```
// Традиционный бесконечный цикл
for {
```

```
    // тело цикла  
}
```

3 – Способы объявления

```
s := ""  
var s string  
var s = ""  
var s string = ""
```

4 – Поиск повторяющихся строк

```
package main
```

```
import (  
    "bufio"  
    "fmt"  
    "os"  
)  
  
func main() {  
    counts := make(map[string]int)  
    input := bufio.NewScanner(os.Stdin)  
    for input.Scan() {  
        counts[input.Text()]++  
    }  
    for line, n := range counts {  
        if n > 1 {  
            fmt.Printf("%d\t%s\n", n, line)  
        }  
    }  
}
```

Про fmt.Printf:

Аттрибут	Описание
%d	Десятичное целое
%x, %o, %b	Целое в hex, oct, bin представлениях
%f, %g, %e	Числа с плавающей точкой: 3.141593, 3.141592653589793, 3.141593e+00
%t	Булево значение: true или false
%c	Руна (символ Unicode)
%s	Строка
%q	Выводит в кавычках строку типа "abc" или символ типа 'c'
%v	Любое значение в естественном формате
%T	Тип любого значения
%%	Символ процента не требует операнда

Простейшая программа которая выводит текст каждой строки, которая появляется во входных данных более одного раза:

```
package main
```

```
import (  
    "bufio"  
    "fmt"  
    "os"  
)  
  
func main() {  
    counts := make(map[string]int)  
    files := os.Args[1:]  
    if len(files) == 0 {  
        countLines(os.Stdin, counts)  
    }  
}
```

```
    } else {
        for _, arg := range files {
            f, err := os.Open(arg)
            if err != nil {
                fmt.Fprintf(os.Stderr, "dup2: %v\n", err)
                continue
            }
            countLines(f, counts)
            f.Close()
        }
    }
    for line, n := range counts {
        if n > 1 {
            fmt.Printf("%d\t%s\n", n, line)
        }
    }
}

func countLines(f *os.File, counts map[string]int) {
    input := bufio.NewScanner(f)
    for input.Scan() {
        counts[input.Text()]++
    }
}
```

5 – Выборка URL

В общем говоря, используя go lang очень легко парсить html...

// Fetch выводит ответ на запрос по заданному URL

```
package main

import (
    "fmt"
    "io/ioutil"
    "net/http"
    "os"
)

func main() {
    for _, url := range os.Args[1:] {
        resp, err := http.Get(url)
        if err != nil {
            fmt.Fprintf(os.Stderr, "fetch: %v\n", err)
            os.Exit(1)
        }
        b, err := ioutil.ReadAll(resp.Body)
        resp.Body.Close()
        if err != nil {
            fmt.Fprintf(os.Stderr, "fetch: чтение %s: %v\n", url, err)
            os.Exit(1)
        }
        fmt.Printf("%s", b)
    }
}
```

6 – Параллельная выборка URL

Важной фишкой Go является поддержка параллельного программирования

```
package main

import (
    "fmt"
    "io"
    "io/ioutil"
    "net/http"
    "os"
    "time"
)

func main() {
    start := time.Now()
    ch := make(chan string)
    for _, url := range os.Args[1:] {
        go fetch(url, ch)
    }
    for range os.Args[1:] {
        fmt.Println(<-ch)
    }
    fmt.Printf("%.2fs elapsed\n", time.Since(start).Seconds())
}

func fetch(url string, ch chan<- string) {
    start := time.Now()
    resp, err := http.Get(url)
    if err != nil {
        ch <- fmt.Sprint(err)
        return
    }
    nbytes, err := io.Copy(ioutil.Discard, resp.Body)
    resp.Body.Close()
    if err != nil {
        ch <- fmt.Sprintf("while reading %s: %v", url, err)
        return
    }
    secs := time.Since(start).Seconds()
    ch <- fmt.Sprintf("%.2fs %7d %s", secs, nbytes, url)
}
```

`go` – это горутини, они представляют собой параллельное выполнение функции. *Канал* является механизмом связи, который позволяет одной горутине передавать значения определенного типа другой горутине.

Функция `main` выполняется в горутине, а инструкция `go` создает дополнительные горутини. Также функция `main` создает канал строк с помощью `make`. Для каждого аргумента командной строки инструкция `go` в первом цикле по диапазону запускает новую горутину, которую `fetch` вызывает асинхронно для выборки URL. При получении каждого результата `fetch` отправляет итоговую строку в канал `ch`, а второй цикл получает и выводит эти строки.

7 – Веб-сервер

Теперь напомним простейший веб-сервер

```
package main

import (
    "fmt"
    "log"
    "net/http"
)

func main() {
    http.HandleFunc("/", handler)
    log.Fatal(http.ListenAndServe("localhost:8000", nil))
}

func handler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "URL.Path = %q\n", r.URL.Path)
}
```

Теперь напишем что-то поинтереснее:

```
package main

import (
    "fmt"
    "log"
    "net/http"
    "sync"
)

var mu sync.Mutex
var count int

func main() {
    http.HandleFunc("/", handler)
    http.HandleFunc("/count", counter)
    log.Fatal(http.ListenAndServe("localhost:8000", nil))
}

func handler(w http.ResponseWriter, r *http.Request) {
    mu.Lock()
    count++
    mu.Unlock()
    fmt.Fprintf(w, "URL.Path = %q\n", r.URL.Path)
}

func counter(w http.ResponseWriter, r *http.Request) {
    mu.Lock()
    fmt.Fprintf(w, "Count %d\n", count)
    mu.Unlock()
}
```

Рассмотрим пример с заголовками и данными

```
func handler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "%s %s %s\n", r.Method, r.URL, r.Proto)
    for k, v := range r.Header {
        fmt.Fprintf(w, "Header[%q] = %q\n", k, v)
    }
    fmt.Fprintf(w, "Host = %q\n", r.Host)
    fmt.Fprintf(w, "RemoteAddr = %q\n", r.RemoteAddr)
    if err := r.ParseForm(); err != nil {
        log.Print(err)
    }
    for k, v := range r.Form {
        fmt.Fprintf(w, "Form[%q] = %q\n", k, v)
    }
}
```

}
}